

**MICROMOUSE: DEVELOPMENT OF A BEHAVIOUR-BASED
ROBOT TO SOLVE A MAZE SCENARIO**

By

**Bandar Alharbi, Dramane Bako, Christopher Eaton, Alex
Senger, Pranav Arya**

Submitted to

The University of Liverpool

MASTER-OF-SCIENCE-ARTIFICIAL-INTELLIGENCE

CSC505 Robotics

Word Count: 1460

14/09/2026

MICROMOUSE: DEVELOPMENT OF A BEHAVIOUR-BASED ROBOT TO SOLVE A MAZE SCENARIO

Submitted to
The University of Liverpool

x

Word Count: 1460

14/09/2026

TABLE OF CONTENTS

	Page
Chapter 1. Introduction	3
Chapter 2. System Design and Implementation	4
2.1 Setting Up the Simulation Environment	4
2.2 Sensing and Perception	5
2.3 Navigation	6
2.4 Algorithmic Approach to Maze Exploration	7
2.5 Mapping and Data Representation	8
Chapter 3. Results and Conclusion	9
3.1 Experimental Results	9
3.2 Discussion	10
3.3 Conclusion	11

LIST OF TABLES

	Page
Table 1 – Comparison of LiDAR and infrared sensing across both mazes	9

LIST OF FIGURES

	Page
Figure 1. The 5x5 Webots simulation arena	4
Figure 2. E-puck robot with LiDAR unit	5
Figure 3. Webots execution console	8
Figure 4. Map outputs generated on the 8x8 complex maze	9
Figure 5. Map output generated on the 5x5 maze using IR proximity probing	9

Chapter 1. INTRODUCTION

Maze-solving robots have been a well-established testbed for behaviour-based robotics, allowing vision, decision-making, and mapping to be tested in a confined environment rather than in open, unstructured navigation. The Micromouse is one in which miniature autonomous robots attempt to solve an unknown maze.

This report describes the design, implementation and evaluation of a behaviour-based system for exploring a bounded grid of walls and mapping the cells it visits. The robot used was an e-puck in a Webots-built 5x5 maze with walls. The robot is unaware of the maze layout; it responds to information from its sensors. The navigation logic is isolated from the sensing hardware, such that sensors can give the same four inputs to the controller. Exploration uses a depth-first search. The path was recorded as the robot moved, and a maze matrix was built to distinguish free cells from walls and from cells not yet visited.

The work was performed in two experiments with the same robot and maze, only the sensor was changed: first, short-range infrared proximity sensors, and then a 360° LiDAR unit. Thus, its scanning plane was beneath the walls of the maze, but clear of the robot's own body. The same exercise was repeated for direct comparison.

The rest of this report covers the environment and robot design, the experimental results, and possible modifications to improve performance.

Chapter 2. SYSTEM DESIGN AND IMPLEMENTATION

This chapter covers how the system was built: the simulation environment, sensing and navigation, the exploration algorithm, and the map representation.

2.1 Setting Up the Simulation Environment

The system was created and tested in Webots R2025a, a simulator that closely replicates rigid-body dynamics, sensor behaviour, and actuation, enabling controller development before physical construction (Michel, 2004). The arena was a $(1.25 \text{ m})^2$ space divided into 5×5 cells, surrounded by 0.10 m walls. A helper module handled the conversion between grid and world coordinates, eliminating the need for the maze logic to be directly aware of geometry.

The robot was an e-puck, a differential-drive platform designed for engineering education and research (Mondada et al., 2009), positioned in the start cell facing east. The world was reset before each run for a clean simulation.

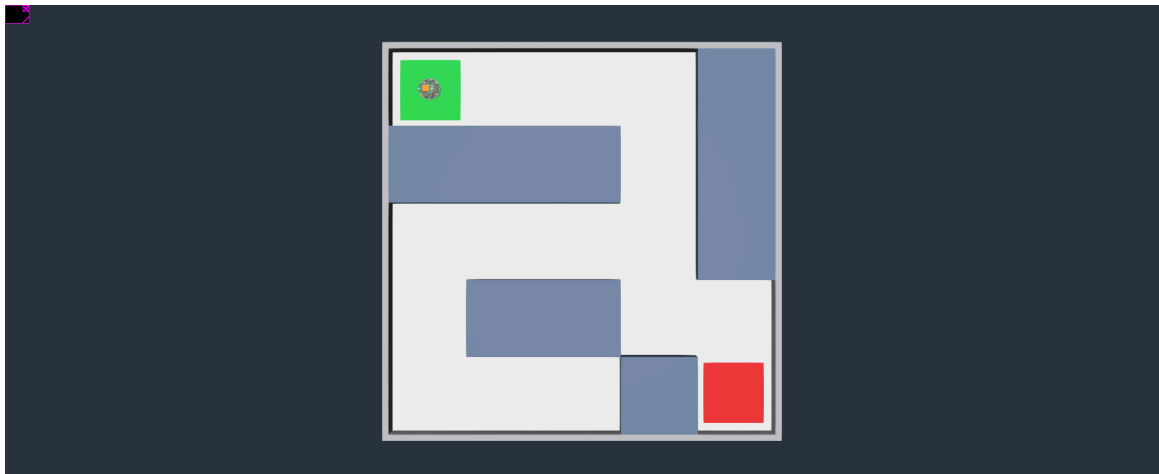


Figure 1. The 5x5 Webots simulation arena showing the e-puck robot starting at the top-left corner.

2.2 Sensing and Perception

Two sensing configurations were tested on the same maze and controller, differing only in wall-detection. The first used eight infrared proximity sensors (ps0–ps7) to infer walls ahead, left, right and behind, versus a fixed threshold. Since IR has a few centimetres of range, a wall a whole cell away is effectively invisible from the centre of the cell. Thus, the robot had to move in a candidate direction and take a spike from a front sensor as confirmation, then reverse back to the centre before trying the next option.

The second setup uses a 360° LiDAR mounted on a turret. This design clears the body from the scan while staying below wall height, allowing it to scan the entire cell centre. By taking the minimum range within four fixed angular sectors, it can detect all neighbouring walls simultaneously without any movement. The distinction between contact and distance sensing is central to this project.

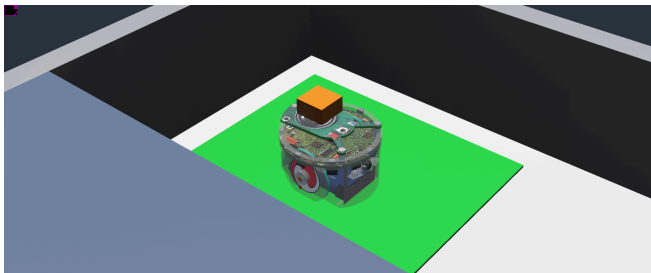


Figure 2. E-puck robot equipped with the top-mounted 360° LiDAR unit.

2.3 Navigation

Movement is a mix of two estimation methods: wheel odometry for distance and a simulated IMU for heading. The distance between cells was measured using wheel encoders, which converted rotations into distance in units of cell size. Closed-loop control maintained heading relative to the IMU: each 90° turn was commanded as an absolute heading from an initial reference yaw to mitigate odometric drift (Siegwart, Nourbakhsh and Scaramuzza, 2011). A proportional controller corrected the side drift when driving straight.

2.4 Algorithmic Approach to Maze Exploration

Exploration used a depth-first search with an explicit backtracking stack, the standard approach when the map is unknown a priori (Cormen et al., 2009). At each cell, the four directions were tried in this order: straight, right, left, back, taking the first open, unvisited neighbour; if none was available, the robot backtracked to the previous cell on the stack. This guarantees the robot will reach the goal if it is reachable, since DFS visits all reachable cells exactly once (Cormen et al., 2009), but it does not guarantee the shortest path.

Both experiments used the same DFS strategy via an abstract four-sided wall interface. LiDAR senses four directions simultaneously from the cell centre, while the infrared sensor uses the front sensors by driving in, checking for contact, and reversing. This allowed the exploration logic to remain constant, so performance differences are due to the sensing method.

2.5 Mapping and Data Representation

As the robot investigated, an occupancy grid was created cell by cell, with each cell sensed as free (0), wall (1), or unexplored (?). LiDAR provided denser maps since one scan filled in all four neighbours of the current cell; IR produced sparser maps as it records walls that were hit during simulation, a fair reflection of what a short-range sensor can know, rather than a defect in the mapping software. The path was logged individually and drawn in red in accordance with the assignment's mapping conventions. Completed maps, step counts, and elapsed time were exported, providing the foundation for the comparison.



Figure 3. Webots execution console showing smoke-test output and sensor readings during the simulation.

2.6 Testing and Validation

System robustness was ensured through an automated pytest suite with 35 tests, all passing, that covered grid unit transformations, DFS state transitions, and occupancy map generation logic, enabling the core maze logic to run independently without the Webots simulator. In addition, visual validation checks were conducted in the Webots simulation console to confirm real-time sensor processing and accurate path tracing.

Chapter 3. Results and Conclusion

3.1 Experimental Results

Both sensors were tested on two mazes: the original 5×5 and an 8×8 "stress test" with a serpentine route and a dead-end trap, using the same starting pose and DFS policy, varying only the sensor.

Table 1: Comparison of LiDAR and infrared sensing across both mazes

Maze	Sensor	Goal reached	Cell steps	Simulated time
5×5	LiDAR	Yes	8	38.8 s
5×5	IR probe	Yes	8	52.4 s
8×8 complex	LiDAR	Yes	32	154.0 s
8×8 complex	IR probe	Yes	32	221.2 s

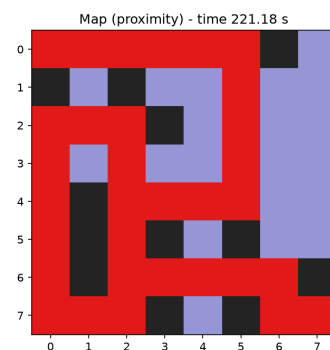
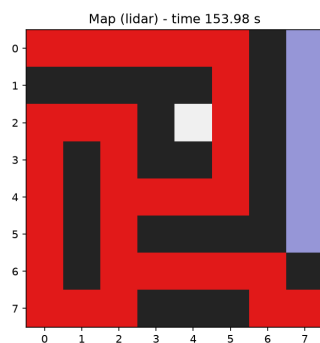


Figure 4. Map outputs generated on the 8x8 complex maze using LiDAR sensing (left) vs IR proximity probing (right). Red indicates the traversed path from the start cell (top-left, row 0, col 0) to the finish cell (bottom-right, row 7, col 7). Black represents detected walls, white indicates free space, and light purple denotes unexplored cells.

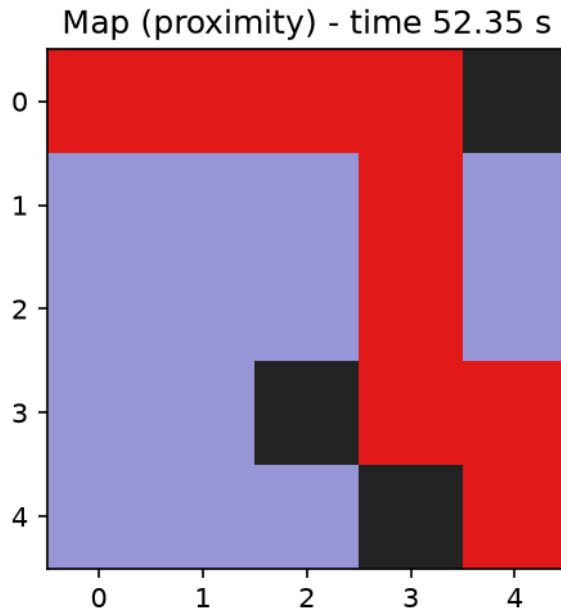


Figure 5. Map output generated on the 5x5 maze using IR proximity probing.

All four runs reached the goal, which confirms that DFS-with-backtracking terminates correctly irrespective of the sensor (Cormen et al., 2009). The number of steps was the same for both sensors on both mazes (8 steps on the 5x5 and 32 on the 8x8), so both sensors recorded the same route, with time being the only variable that differed. IR took 35.1% longer on the 5x5 maze and 43.6% longer on the 8x8 maze, a consistent overhead matching the extra drive-in-bump-reverse step needed for every wall IR discovered, which LiDAR avoids by scanning from the centre. The maps themselves were also different: LiDARs were dense, in that all four neighbours were filled in at every visited cell, while IRs were noticeably sparser, in that only bumped walls were recorded (as illustrated in Figure 4 for the 8x8 maze).

3.2 Discussion

The results indicate a distinct trade-off between sensor choice and exploration speed, with planning held fixed. LiDAR reads all four walls from a single stationary scan, exactly matching IR's route but taking 26.0% less time on the 5×5 and 30.4% less time on the 8×8, consistent with the general range-vs-cost trade-off known for exteroceptive sensors (Siegwart, Nourbakhsh and Scaramuzza, 2011). Identical step counts with different completion times show that the gap comes from the physical cost of gathering information, not from either sensor producing worse decisions, something testable due to the abstract wall interface that separates planning from sensing.

This work can be further developed with some improvements. Having built the map, a Dijkstra pass would enable a faster, optimised run rather than the straight-right-left-back rule; a graded IR distance reading, rather than a binary bump threshold, could detect walls before contact, removing the reverse-and-recentre overhead that accounts for most of the extra time. A key limitation is that localisation relies on dead reckoning without external ground-truth correction. Heading drift is managed via the simulated IMU, but uncorrected wheel slip during translation may accumulate error on larger mazes.

3.3 Conclusion

In this project, a DFS-driven maze exploration e-puck robot system was evaluated in Webots, comparing LiDAR and IR sensing across 5×5 and 8×8 mazes. Both configurations successfully solved both test mazes using the same DFS strategy: the 5×5 took 8 moves, and the 8×8 took 32 moves, against a shortest possible of 20. IR took 35.1% longer than LiDAR in the 5×5 maze and 43.6% in the 8×8 maze due to the physical probing required by short-range sensors. While these results show that sensor choice directly affects exploration speed in the tested setups, further validation in larger environments and on physical hardware is required before generalising across all parameters.

REFERENCES

Cormen, T.H., Leiserson, C.E., Rivest, R.L. & Stein, C. (2009). *Introduction to Algorithms*. 3rd edn. Cambridge, MA: MIT Press.

Michel, O. (2004). 'Webots: Professional mobile robot simulation', *International Journal of Advanced Robotic Systems*, 1(1), pp. 39–42.

Mondada, F., Bonani, M., Raemy, X., Pugh, J., Ciani, C., Klapacz, A., Magnenat, S., Zufferey, J-C., Floreano, D. and Martinoli, A. (2009). 'The e-puck, a robot designed for education in engineering', *Proceedings of the 9th Conference on Autonomous Robot Systems and Competitions*, 1(1), pp. 59–65.

Siegwart, R., Nourbakhsh, I.R. & Scaramuzza, D. (2011). *Introduction to Autonomous Mobile Robots*. 2nd edn. Cambridge, MA: MIT Press.

APPENDICES

Appendix A. APPENDIX A: PROJECT REPOSITORY

Project Repository

<https://github.com/Bandarbinsaad/csck505-group-project>